MASTER VIVA PREPARATION & TECHNICAL DEFENSE GUIDE

Intelligent Task Allocation and Scheduling System with ML-Assisted Optimization

Team Byte_hogs: Shri Srivastava (2023ebcs593), Ichha Dwivedi (2023ebcs125), Aditi Singh (2023ebcs498)
Course: BCS ZC241T Study Project | **Supervisor:** Swapnil Saurav | **Institution:** BITS Pilani Online

---

1. The 60-Second & 3-Minute Master Pitches

■ ***60-Second Elevator Pitch:***
"Good morning, esteemed examiners. Our project, ML Task Scheduler, is an enterprise-grade distributed platform for multi-tiered Fog-Cloud architectures. In real-world IoT systems, allocating heterogeneous tasks to edge nodes is an NP-hard problem where traditional heuristics fail because they assume static execution times. We extend the mathematical optimization model of Wang & Li (2019), integrating Bio-Inspired Metaheuristics (IPSO, IACO, Hybrid Heuristic), Deep Reinforcement Learning (MaskablePPO), and Machine Learning Regressors (XGBoost/Random Forest, $R^2 = 0.9483$) with sub-15ms latency. Built with React 18, Express, PostgreSQL (30 models), Redis, and BullMQ worker pools, the platform delivers real-time scheduling in under 200ms with SHAP explainability and 90% conformal uncertainty intervals."

2. Core Technical Viva Questions & Defenses

Q1: What problem does your project solve?

Answer: We solve the multi-objective, NP-hard task-to-resource allocation problem across heterogeneous Fog-Cloud tiers, minimizing latency and terminal device energy while strictly respecting deadline SLAs.

Q2: Why is Machine Learning needed instead of simple rules?

Answer: Simple rules assume linear execution (Time = DataSize / Capacity). In real systems, non-linear factors like CPU cache contention, task priority preemption, startup overhead, and concurrent load cause over 40% error. Our ML model predicts true execution with $R^2 = 0.9483$.

Q3: What makes your Hybrid Heuristic (HH) better than pure PSO or ACO?

Answer: IPSO provides rapid global swarm exploration but suffers from premature convergence. IACO provides fine-grained local path refinement but wanders blindly initially. Our HH runs IPSO to find a promising neighborhood and seeds IACO's initial pheromone matrix (τ_0), speeding up convergence by 35%.

Q4: What is Split Conformal Prediction, and why is it essential?

Answer: Point predictions give no measure of certainty. Split Conformal Prediction is a distribution-free uncertainty quantification technique. Calibrating on held-out residuals gives a quantile bound q_hat guaranteeing that true execution times fall in [y_hat - q_hat, y_hat + q_hat] with exactly 90% probability ($\alpha = 0.10$).

Q5: What happens if the Python ML microservice crashes?

Answer: We implemented a Circuit Breaker and Graceful Fallback pattern in Express. If the ML service fails (503/timeout), the system logs a warning and automatically falls back to deterministic heuristic duration calculation without crashing the API or dropping user requests.

Q6: Why did you choose PostgreSQL with Prisma over MongoDB?

Answer: Task scheduling requires strict ACID transactions. Updating task statuses, incrementing fog node loads, and logging schedule history must occur atomically ($transaction). Relational foreign keys and cascading integrity prevent orphaned tasks or phantom resource over-allocations.

Q7: How do you prevent CSRF and Session Hijacking?

Answer: We implement the Double-Submit Cookie Pattern: a cryptographically signed cookie is matched against the X-CSRF-Token HTTP header on all mutating requests (POST/PUT/DELETE). Access tokens are short-lived (15 min) and refresh tokens use single-use token rotation stored in PostgreSQL.

Q8: What are the 4 asynchronous worker queues?

Answer: BullMQ + Redis queues: (1) prediction.worker.ts for batch ML inference, (2) scheduling.worker.ts for metaheuristic calculations, (3) notification.worker.ts for WebSocket broadcasts, and (4) autoRetrain.worker.ts for background model retraining with Dead Letter Queues (DLQ).

3. 10-Step Live Demonstration Script

Step	Action	Technical Explanation to Deliver
1	Open http://localhost:3000	Explain landing dashboard, architecture overview, and 3-tier fog topology.
2	Login (admin@example.com)	Show JWT authentication, HTTP-only cookie issuance, and RBAC admin role.
3	Create Task (/tasks)	Create a LARGE CPU task with priority 5; explain client-side Zod validation.
4	Show ML Prediction Card	Highlight sub-15ms predicted duration, SHAP waterfall breakdown, and 90% conformal bounds.
5	Navigate to Fog Topology	Show 10 interactive Fog Nodes with dynamic SVG color coding based on active load.
6	Execute Hybrid Heuristic	Run HH algorithm; explain IPSO swarm initialization and IACO pheromone refinement.
7	Run Benchmark (/experiments)	Execute Wang & Li (2019) Figure 5 test; show generated delay and energy comparative curves.
8	Download PDF Report	Generate and download PDFKit executive scheduling summary report with embedded charts.
9	Inject Chaos Latency (/chaos)	Inject 500ms delay; demonstrate circuit breaker fallback and system resilience.
10	Show Grafana (:3002)	Display live Prometheus request rates, Redis queue latencies, and memory metrics.

4. Top Examiner Traps & Bulletproof Defenses

■■ Examiner Trap: 'Does 90% conformal coverage mean 90% accuracy?'

DEFENSE: 'No, professor. Conformal coverage is an uncertainty guarantee under exchangeability: it means that the true execution duration will mathematically fall within the calculated confidence interval [y_hat - q_hat, y_hat + q_hat] exactly 90% of the time, regardless of the underlying data distribution.'

■■ Examiner Trap: 'Your project is just a CRUD application with React and Node.js.'

DEFENSE: 'While we provide a full-stack interface, the core intellectual contribution is the mathematical optimization engine in backend/src/services/fog/ which solves an NP-hard scheduling problem via two-stage metaheuristics and sub-15ms ML regressors, validated through Wang & Li (2019) benchmark reproductions.'

■■ Examiner Trap: 'Why not use Deep Learning / Transformers for execution prediction?'

DEFENSE: 'For tabular data with 5 dense numerical/categorical features and low sample sizes (<100k), tree ensembles (XGBoost, Random Forest) achieve higher accuracy ($R^2 = 0.9483$), train in seconds on CPU, and execute in under 5ms, avoiding the 50ms+ GPU latency overhead of Transformers.'